```
var_dump(microtime());
var_dump(microtime(true));
```

```
Output:
string(21) "0.84775700 1738869451"
float(1738869451.847781)
```

`microtime()` accepts a single boolean argument (which defaults to `false`) and returns either a string containing milliseconds and seconds since the Epoch or the number of seconds elapsed as a float (the decimal portion providing enough precision for millisecond resolution). By far the most common use of `microtime()` is for diagnosing performance problems: a common technique is to wrap potentially time-costly calls with `microtime()` like so:

```
$start = microtime(true);
sleep(2);
$end = microtime(true);
print "Time elapsed : " . $end - $start . "s";
```

```
Output:
Time elapsed : 2.0040378570557s
```

In an environment where entire requests might need to meet SLAs measured in fractions of seconds, `microtime()` can be a very handy quick-and-dirty way to narrow down where to focus optimization effort.

Summary

Hopefully you now have an appreciation of the challenges presented by dealing with time in your PHP code and the tools available to address them. These problems are often exacerbated by the nature of PHP's typical environment: web applications. PHP code often has to interact with remote systems: the timezone of your web server may not exactly match that of a third-party API hosted in another country, and HTTP clients could be anywhere in the world (and one day maybe not even limited to Earth: there are already calls in the scientific community for a lunar timezone). There is also the tricky problem of managing daylight savings: those times and dates where clocks are changed to "better" suit a particular day/night cycle.